T1 纪念品分组（souvenir）

排序后处理最贵的一件

每袋最多装两件，因此可以先决定剩余纪念品中最贵的一件怎样装袋。把价格升序排列，用 l, r 指向尚未装袋的最便宜和最贵纪念品。

若 $a_l + a_r > w$，连最便宜的一件都不能与 a_r 同袋，a_r 只能单独装。

若 $a_l + a_r \le w$，可以让两者同袋。理由是：取一个最优方案，若两者已有不同搭档 x, y，原来 $a_r + y \le w$，又有 $x \le a_r$，故交换成 (a_l, a_r) 和 (x, y) 后两袋仍合法；若其中一件单独成袋，配在一起后让原搭档单独装，也不会增加袋数；若两件都单独装，合并反而少用一袋。因此总能把最优方案调整成当前选择。

每轮用一袋装走最贵者；能配对时同时移走最便宜者，否则只移走最贵者。剩一件时单独装。以上选择每次都能保留一个最优方案，处理完所有物品时，累计袋数就是最少袋数。

排序耗时 $O(n \log n)$，双指针共移动 $O(n)$ 次，总时间 $O(n \log n)$；代码保存价格数组，空间 $O(n)$。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 30000 + 10;
int n,w,a[N];

int main(){
    freopen("souvenir.in","r",stdin);
    freopen("souvenir.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> w >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];
    sort(a + 1,a + n + 1);
    int l = 1,r = n,ans = 0;
    while(l <= r){
        if(l < r && a[l] + a[r] <= w){
            l++;
            r--;
        }else{
            r--;
        }
        ans++;
    }
    cout << ans << '\n';
    return 0;
}
```

部分分

每点 5 分；表中为题面点组分值。

序号	条件	点号与分值	做法与复杂度
1	$n \le 15$	$1 \sim 8$，40 分	用集合状态记录未装物品，固定其中编号最小者，枚举它单独装袋或与另一件合法配对，记忆化最少袋数。空集为 0。时间 $O(n2^n)$，空间 $O(2^n + n)$。
—	$n \le 1000$	$9 \sim 12$，20 分	按正文排序，逐次处理最贵者，能配对时取最便宜者。时间 $O(n \log n)$，空间 $O(n)$。
—	无额外限制	$13 \sim 20$，40 分	同一排序贪心覆盖完整范围，时间 $O(n \log n)$，空间 $O(n)$。

T2 安全密码（password）

让每个前缀尽量晚地匹配完

判断一个固定密码是否为子序列时，应从左到右匹配，每位都取能找到的最早位置。这样留给后缀的字符最多；如果最早匹配都失败，换成更晚的位置也不会成功。

本题要找一个匹配失败的密码，所以每增加一位，都希望让这个前缀的**最早匹配结束位置尽量靠右**。结束位置越晚，剩余字符串越短，后面越容易匹配失败。

沿用代码的 0 下标，令 $\mathsf{nxt}[d][i]$ 表示从位置 i 起数字 d 第一次出现的位置，不存在时记为 $n = |s|$。从右向左建表：先复制位置 $i+1$ 的十个答案，再把数字 s_i 的答案改成 i；边界为 $\mathsf{nxt}[d][n] = n$。

设 cur 是当前所有允许前缀的最早匹配结束位置中的最大值，初始为 -1。处理密码第 i 位时，对每个允许数字 d 查询

$$p_d = \mathsf{nxt}[d][\mathsf{cur}+1].$$

若某个 $p_d = n$，选取实现 cur 的前缀，再接上 d，已经无法匹配；后面任意补上允许数字即可，答案为 YES。

否则令 $\mathsf{cur} = \max p_d$，继续下一位。这个更新确实保留了所有前缀中最晚的结束位置：对固定数字 d，从越靠右的位置开始找，它的首次出现位置只会更晚；而旧的 cur 本身可以由某个允许前缀达到。因此其余前缀不会产生比本轮更晚或更早失败的分支。若处理完 m 位仍未失败，所有允许密码都能匹配，输出 NO。

单组建表和判断的时间为 $O(10n+10m)$，空间为 $O(10n+m)$；所有组总时间为 $O(10\sum n+10\sum m)$。本题 $\sum n \le 300000$，$m \le 10$。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 300000 + 5;
int nxt[10][N];

int main(){
    freopen("password.in","r",stdin);
    freopen("password.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while(t--){
        string s,l,r;
        int m;
        cin >> s;
        cin >> m;
        cin >> l >> r;
        int n = (int)s.size();
        for(int d = 0; d < 10; d++) nxt[d][n] = n;
        for(int i = n - 1; i >= 0; i--){
            for(int d = 0; d < 10; d++) nxt[d][i] = nxt[d][i + 1];
            nxt[s[i] - '0'][i] = i;
        }
        int cur = -1;
        bool found = false;
        for(int i = 0; i < m && !found; i++){
            int far = -1;
            for(int d = l[i] - '0'; d <= r[i] - '0'; d++){
                int p = nxt[d][cur + 1];
                if(p == n){
                    found = true;
                    break;
                }
                far = max(far,p);
            }
            cur = far;
        }
        cout << (found ? "YES" : "NO") << '\n';
    }
    return 0;
}
```

部分分

每点 5 分。第 1 项程序同时覆盖两个互不重叠的点组，保证 25 分。

序号	条件	点号与保证分	做法与复杂度
1	$m \le 3$，且总字符串长 ≤ 1000	$1 \sim 4$，20 分	枚举所有允许密码，从左向右扫描 s 判断子序列，找到一个失败者即输出 YES。若候选数为 $P = \prod_i (r_i - l_i + 1)$，单组时间 $O(P(n+m))$，空间 $O(n+m)$。
复用 1	每位只允许一个数字	20，5 分	候选只有一个，直接扫描判断。上项程序的 $P = 1$，时间 $O(n+m)$。
—	每组 $n \le 1500$	$5 \sim 8$，20 分	用正文的后继表和最晚前缀匹配，单组时间 $O(10n + 10m)$，空间 $O(10n + m)$。
—	总字符串长 ≤ 100009 或完整范围	$9 \sim 19$，55 分	沿用同一满分方法，按各组长度之和计算总时间；无需枚举允许密码。

T3 数组升格（growmax）

二分目标值，计算右侧的支撑成本

一次操作只增加 1，所以答案在原最大值 A 与 $A+k$ 之间。能把某个元素升到 H，就一定能使最大值至少达到更小的目标；可行性随 H 增大只会变差，可以二分最后可行的值。

检查 H 时，枚举希望升到 H 的位置 l。若 $a_l < H$，最后一次增加 a_l 前，它等于 $H-1$，右邻居至少也要达到 $H-1$。若右邻居原值不足，又需要再右边达到 $H-2$。由操作限制，向右所需的高度依次为

$$H,\ H-1,\ H-2,\ldots.$$

从 l 向右扫描，用 need 表示当前位置所需高度。若原值已满足 $a_i \geq$ need，这条支撑链可以在这里结束；否则必须支付 need $- a_i$ 次增加，再把 need 减一。若不足的位置已经是 n，它没有右邻居，不能再增加，这个起点不可行。累计成本超过 k 时也可以立即停止。

这些缺口既必要，也足够：必要性来自每个位置最后一次增加时的右邻居限制；充分性则是先完成链上最右边的缺口，再向左依次增加，各次操作的右邻居都已达到所需高度。因此在支撑链能够结束的前提下，缺口总和就是最少操作数。存在一个成本不超过 k 的起点，目标 H 就可行。

代码用 low=A 表示已知可行值，hi=A+k+1 表示不可行上界，循环检查中点，最后输出 low。一个起点最多扫描 n 个位置，一次判定 $O(n^2)$；二分次数 $O(\log(k+1))$，单组时间 $O(n^2\log(k+1))$、空间 $O(n)$。所有组满足 $\sum n \leq 1000$，故 $\sum n^2 \leq 10^6$，至多约 27 轮判定。成本累加使用 long long。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 1000 + 10;
int n;
long long k,a[N];

bool possible(long long target){
    for(int l = 1; l <= n; l++){
        long long need = target;
        long long cost = 0;
        bool ok = true;
        for(int i = l; i <= n; i++){
            if(a[i] >= need) break;
            if(i == n){
                ok = false;
                break;
            }
            cost += need - a[i];
            if(cost > k) break;
            need--;
        }
        if(ok && cost <= k) return true;
    }
    return false;
}

int main(){
    freopen("growmax.in","r",stdin);
    freopen("growmax.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while(t--){
        cin >> n >> k;
        long long hi = 0;
        for(int i = 1; i <= n; i++){
            cin >> a[i];
            hi = max(hi,a[i]);
        }
        long long low = hi;
        hi += k + 1;
        while(low + 1 < hi){
            long long mid = (low + hi) / 2;
            if(possible(mid)) low = mid;
            else hi = mid;
        }
        cout << low << '\n';
    }
    return 0;
}
```

部分分

每点 5 分；前三个点组互不重叠，组合覆盖 60 分。

序号	条件	点号与保证分	做法与复杂度
1	$n \le 5, k \le 6$	$1 \sim 4$，20 分	每步枚举所有能加一的位置，递归后撤销；在任意深度用当前最大值更新答案，允许提前停止。时间上界 $O(n(n-1)^k)$，空间 $O(n+k)$。
2	$n = 2$	$5 \sim 8$，20 分	右项不能改变。若 $a_1 > a_2$，答案为 a_1；否则左项最多增加 $\min(k, a_2 - a_1 + 1)$ 次，再与 a_2 比较。读取后计算 $O(1)$。
复用 2	严格递减	$9 \sim 12$，20 分	没有合法操作，输出 a_1。第 2 份源码按结构分派，两档时间均为 $O(n)$、空间 $O(n)$，计入读入与保存数组。
—	无额外限制	$13 \sim 20$，40 分	正文二分目标并计算支撑链成本，单组 $O(n^2 \log(k+1))$ 时间、$O(n)$ 空间。

T4 广阔图（widegraph）

非孤立点能否属于同一个连通块

G_k 只连接树上距离至少为 k 的点对。若某点到所有点的距离都不足 k，它必然孤立；其余点怎样连通，可以从树上相距最远的两个点入手。

设 s,t 是直径端点，$D=d(s,t)$。对任意点 v，它到最远点的距离满足

$$e(v)=\max\big(d(v,s),d(v,t)\big).$$

说明这个性质：把直径当作长度为 D 的主路径，设 v 接入主路径的位置距 s 为 x，接入距离为 h。另一个点 u 接入主路径的位置距 s 为 y，接入距离为 g。直径最长，故 $g\le\min(y,D-y)$。若 $y\le x$，则

$$d(v,u)\le h+x-y+g\le h+x=d(v,s);$$

若 $y\ge x$，同理有 $d(v,u)\le h+D-x=d(v,t)$。路径共享一段支路时，上式第一步仍是合法上界。两个端点本身都能到达，便得到偏心距公式。

当 $k\le D$，s,t 在 G_k 中直接相连。只要 $e(v)\ge k$，上式保证 v 至少与其中一个端点相连，所以全部非孤立点都在这个连通块里；$e(v)<k$ 的点各自成为一个孤立块。因此

$$\text{ans}_k=\begin{cases}1+\#\{v:e(v)<k\}, & k\le D,\\ n, & k>D.\end{cases}$$

取得直径端点可用两次 BFS：任取一点，找最远点 s，再从 s 找最远点 t。这里也能用刚才的公式说明第一步：对任意一条直径，任一点的最远距离由其两端之一达到；若 BFS 选到了并列的其他最远点，它也能与某个直径端点组成等长直径。因此任取最远点都可作为直径端点。再从 t 做一次 BFS，便有两份距离数组。

计算所有 $e(v)$ 并升序排序。对每个 k，用 lower_bound 找第一个不小于 k 的值，它之前的个数就是孤立点数。三次 BFS 共 $O(n)$，排序和全部查询共 $O(n\log n)$；总时间 $O(n\log n)$，邻接表和数组占 $O(n)$ 空间。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 100000 + 10;
int n,dist1[N],dists[N],distt[N],ecc[N];
vector<int> e[N];

void bfs(int s,int dist[]){
    for(int i = 1; i <= n; i++) dist[i] = -1;
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while(!q.empty()){
        int u = q.front();
        q.pop();
        for(int i = 0; i < (int)e[u].size(); i++){
            int v = e[u][i];
            if(dist[v] == -1){
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
```

```
    }
}

int farthest(int dist[]){
    int p = 1;
    for(int i = 2; i <= n; i++){
        if(dist[i] > dist[p]) p = i;
    }
    return p;
}

int main(){
    freopen("widegraph.in","r",stdin);
    freopen("widegraph.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n;
    for(int i = 1; i < n; i++){
        int u,v;
        cin >> u >> v;
        e[u].push_back(v);
        e[v].push_back(u);
    }
    bfs(1,dist1);
    int s = farthest(dist1);
    bfs(s,dists);
    int t = farthest(dists);
    bfs(t,distt);
    for(int i = 1; i <= n; i++){
        ecc[i] = max(dists[i],distt[i]);
    }
    sort(ecc + 1,ecc + n + 1);
    int diameter = ecc[n];
    for(int k = 1; k <= n; k++){
        if(k > diameter){
            cout << n;
        }else{
            int isolated = (int)(lower_bound(ecc + 1,ecc + n + 1,k) - ecc - 1);
            cout << 1 + isolated;
        }
        cout << (k == n ? '\n' : ' ');
    }
    return 0;
}
```

部分分

每点 5 分。第 2 份源码同时处理链和菊花；这两个点组分别计分。

序号	条件	点号与保证分	做法与复杂度
1	$n \le 10$	$1 \sim 4$，20 分	从每个点遍历树求距离表，对每个阈值显式检查所有点对，以并查集合并满足条件者。使用仅路径压缩的并查集，保守时间上界 $O(n^3 \log(n+1))$，空间 $O(n^2)$。
2	树是一条链	$5 \sim 8$，20 分	从端点求每点的链上位置 x，偏心距为 $\max(x, n-1-x)$；排序后按正文公式计数。时间 $O(n \log n)$，空间 $O(n)$。
复用 2	树是菊花图	$9 \sim 12$，20 分	$n \ge 3$ 时，$k = 1, 2, \ge 3$ 的块数分别是 $1, 2, n$。$n = 2$ 也是链，由链分支处理。总时间、空间 $O(n)$。
—	一般树	$13 \sim 20$，40 分	三次 BFS、偏心距排序与阈值计数，时间 $O(n \log n)$，空间 $O(n)$。